m1

Three.js Terrain Sculpting System | Smooth Brush, Dynamic Height Coloring, Reset Map & Environmental Props Engine

1. Overview

This module describes an advanced **interactive terrain editing system built with Three.js**, focusing on real-time mesh deformation, procedural styling, and environmental interaction tools.

The system transforms a static 3D terrain into a **fully interactive world editor** where users can sculpt landscapes dynamically using brush-based tools and real-time geometry updates.

https://www.udemy.com/course/web-based-3d-terrain-industrial-printing-engine-mastery/

2. Terrain Sculpting Tools

The engine includes three primary terrain manipulation modes:

Raise Tool

Adds height to selected vertices based on brush radius and falloff curve.

Dig Tool

Subtracts height from terrain vertices to create valleys and depressions.

Smooth Tool (New Feature)

Instead of modifying height directly, this mode:

- Gradually moves each vertex toward the average

This produces natural terrain smoothing effects by eliminating sharp edges and jagged formations.

3. Height-Based Dynamic Coloring System

Terrain visual realism is enhanced using elevation-based coloring rules:

- **Low elevation (valleys):** dark green / blue tones
- **Mid elevation:** earthy brown tones
- **High elevation (Y > 25):** white (snow peaks)

This system can be implemented using either:

- Vertex colors (BufferGeometry color attribute)
- Shader-based fragment coloring logic

The result is a procedurally colored terrain that visually communicates elevation structure.

4. Reset Map System

A full terrain reset feature allows users to restore the landscape to a flat baseline.

Function behavior:

- Iterate through all vertices in geometry
- Set all Y values to 0
- Mark position attribute as updated
- Recompute normals
- Refresh rendering pipeline

This ensures the terrain returns to a clean editable state without reloading the scene.

5. Environmental Prop System (Bonus Feature)

The system supports object placement on terrain using raycasting interaction.

Behavior:

- Hold specific key (e.g., **T for Tree**)

- Spawn primitive object (cone or cylinder)
- Align object position to terrain height

This enables procedural world decoration such as:

- Trees
- Rocks
- Pillars
- Environmental markers

6. Raycasting Mechanics (Core Concept)

Raycasting is used to convert 2D mouse coordinates into a 3D interaction point.

Process:

- Convert mouse position → Normalized Device Coordinates (NDC)
- Project ray from camera into 3D space
- Detect intersection with terrain mesh

This is required because 3D environments cannot directly interpret 2D screen input without spatial projection.

7. Geometry Update System

When terrain vertices are modified:

- `position.needsUpdate = true` is required

If omitted:

- GPU will not receive updated vertex buffer
- Visual terrain remains unchanged
- Internal data ≠ rendered output

This is critical for real-time mesh deformation systems.

8. Falloff Mathematics & Terrain Shape Control

Brush falloff determines how influence decreases from center outward.

- Cosine falloff → smooth organic hills
- Linear falloff → sharper, artificial slopes

- Natural terrain vs mechanical shaping

9. Performance Optimization (Scalability)

Large terrain grids (e.g., 512x512) introduce performance challenges.

Optimization strategies:

- Spatial partitioning (quadtrees / grid indexing)
- Vertex pre-filtering
- Brush bounding box checks
- GPU-based deformation (advanced)
- Reduced iteration radius lookup

Goal: avoid scanning entire vertex array on every mouse move.

10. Lighting & Normal Recalculation

`computeVertexNormals()` is required after every terrain modification.

If removed:

- Lighting becomes incorrect
- Terrain appears flat or broken
- Shadows do not align with geometry
- Surface detail is visually lost

Normals define how light interacts with surfaces, making them essential for realistic rendering.

11. Learning Objectives

By completing this module, students will be able to:

- Build full Three.js application architecture
- Implement interactive terrain sculpting tools
- Manipulate BufferGeometry in real time
- Use Raycaster for 3D interaction systems
- Apply falloff mathematics for procedural shaping
- Implement height-based dynamic material systems
- Optimize performance for large-scale terrain grids

m2
m3

© 2026 Okan Kaplan – MIT License Read full license